Duration:

5 days

Prerequisites:

This is NOT a beginner course, rather intermediate to advanced level course. This course is explicitly for those who are already working on .NET Framework or Core projects, i.e., for the participants

a. who have working knowledge on .NET Framework and/or .NET Core and
b. who are familiar with topics such as *basics of C#, OOP in C#, inheritance, polymorphism, interface, abstract classes, exception handling, collections and generic collections* etc. along with basics of unit testing.

Objective:

After this course participants will be able to

a. Learn about .NET (previously known as .NET Core) and its components
b. Differences between .NET and legacy .NET Framework
c. Dependency Injection .NET Core
d. Revisit generics, reflection and delegates in depth
e. Learn how to use delegates in case of event handling or Lambda expression
f. Learn how to threads and Tasks
g. Learn about parallel Programming
h. Learn about EF Core for data access and how to use LINQ against EF Core data sources
i. Learn about ASP.NET Core Web API

Content with day-wise break-up:

Day-1:

1. Introduction to .NET Core
    a. What is .NET Core or .NET?
    b. .NET Core/.NET vs. .NET Framework
    c. How to create an application from scratch
    d. Modularity in .NET Core/.NET
    e. NuGet Packages

2. Deep Dive into .NET Core
    a. Understand the details of application and its execution
    b. Different files that are required for running

c. Dependency configuration file, Runtime configuration file etc.
d. Using external packages in your application
e. Understand execution in detail
f. How Cross-platform advantage is provided?
g. Components of .NET:
    a. Runtime
    b. Libraries
    c. Compiler
    d. SDK and other tools
    e. App Stacks
h. Runtimes of .NET
i. Roslyn compiler of .NET
j. .NET CLI: dotnet
k. dotnet.exe and it's commands
l. .NET Standard: is it a framework?
m. Understanding .NET Native
n. Hosting .NET Core/.NET application: Really Host is required for non-web applications too?
o. .NET Core Versions: 1.0 - 9.0
p. Future: .NET 10

3. Delegates

a. Introduction to delegate
b. Internals of delegate
c. Instance and Static Method Targets
d. Multicast Delegate
e. Generic Delegate Types
f. The Func, Predicate and Action Delegates
g. Covariance
h. Contravariance

4. New features in C#

a. Implicitly typed local variables
b. Anonymous Types
c. Extension Methods
d. Object and Collection Initializer
e. Lambda Expressions
f. Query Expressions
g. Named and Optional Parameters
h. Asynchronous members
i. Static imports
j. Property initializers
k. Expression bodied members
l. Null propagator
m. String interpolation
n. nameof operator
o. out variables
p. Tuples
q. Local functions

r. Readonly members
s. Default interface methods
t. Switch expressions
u. Property patterns
v. Static local functions
w. Nullable reference types
x. Null-coalescing assignment

Day-2

5. LINQ

a. What is LINQ?
b. LINQ syntax: Method Query syntax and Query Operator syntax
c. LINQ Query Operator Syntax
d. Declaring a Variable for Results Using the var Keyword
e. Specifying the Data Source: from Clause
f. Specify Condition: where Clause
g. Selecting Items: select Clause
h. Finishing Up: Using the foreach Loop
i. Deferred Query Execution
j. LINQ Method Syntax
k. LINQ Extension Methods
l. Query Syntax versus Method Syntax

6. Entity Framework core

a. Basic Workflow in Entity Framework
b. How Entity Framework Works?
c. Entity Framework Architecture
d. Context Class in Entity Framework
e. What is an Entity in Entity Framework?
f. Scalar Property
g. Navigation Property
h. Reference Navigation Property
i. Collection Navigation Property
j. POCO Entities (Plain Old CLR Object)
k. Development approach: Code First and DB First Approach
l. DbContext
m. Creating first application
n. Querying
o. Saving data
p. Conventions
q. Configurations
r. Fluent API
s. Working with stored procedure
t. Logging
u. Migration

Day-3

7. Dependency Injection in .NET

a. Dependency Injection
b. DI through interface
c. DI in .NET
d. ServiceCollection
e. ServiceProvider
f. Build service provider
g. Scope and create scope
h. Lifetime of services: Singleton, Scoped and Transient

8. Configuration in .NET

a. Configuration in .NET application
b. Configuration provider
c. File configuration provider
d. Environment variable configuration provider
e. Command line configuration provider
f. Key-per-file configuration provider
g. Using appsettings.json file
h. Reading from appsettings.json file
i. Options Pattern
j. Using options pattern to read from json file

9. Logging in .NET

a. Logging service
b. Logging providers
c. Using logging service in your application

10..NET Generic host

a. What is hosting in.NET?
b. Responsibilities of a host
c. Set up a host
d. Host builder settings
e. Framework provided services
f. Host application lifetime

Day-4

11. Threading and Tasks in C#

a. Threads in C#
b. Multithreading
c. Thread class
d. Create a thread and Start a Thread
e. Synchronization of thread using Thread.Join, Lock and Monitor
f. Tasks
g. Starting Task and Results from Tasks
h. Continuing Tasks

12.ASP.NET Core Application Architecture

a. What is ASP.NET Core and its advantages
b. What are new features here?
c. What is the difference with ASP.NET Framework
d. Create an ASP.NET Core application from scratch
e. In Visual Studio Empty project will be used
f. In Visual Studio Code “web” type application will be used
g. Main method in Web
h. Are we creating console or web application?
i. Hosting Environment
j. WebApplication (web host) and WebApplicationBuilder
k. Which web server is hosting?
l. Kestrel, IIS Express, IIS, Apache etc.
m. Application Startup (Understanding .NET Core Application Execution) code

13. Dependency Injection in ASP.NET Core

a. Overview of dependency injection
b. Register groups of services with extension methods
c. Service lifetimes
d. Service registration methods
e. Constructor injection behavior
f. Entity Framework contexts
g. Lifetime and registration options
h. Resolve a service at app start up
i. Scope validation
j. Request Services
k. Design services for dependency injection
l. Default service container replacement
m. Recommendations
n. Recommended patterns for multi-tenancy in DI
o. Framework-provided services

14. Startup and Middleware

a. Request Processing Pipeline
b. Introduction to Middleware
c. Why middleware?
d. Middleware code analysis
e. Create a middleware pipeline with WebApplication
f. Different middleware
g. For Exception handling
h. For Static Files
i. For environment files: UseCors and UseStaticFiles order
j. Middleware order
k. Branch the middleware pipeline
l. Built-in middleware
m. Creating Custom middleware

Day-5

15. Use HttpContext in ASP.NET Core

a. HttpRequest

b. HttpResponse
c. RequestAborted
d. Abort()
e. User
f. Features
g. HttpContext isn't thread safe

16. Request Routing
a. Routing basics
b. Routing concepts
c. Route templates
d. Routing with special characters
e. Route constraints
f. Parameter transformers

17. ASP.NET Core 8 Web APIs
a. Introduction to ASP.NET Core Web API
b. Types of Web APIs
c. Controller-based APIs
d. Minimal APIs

18. Controller-based APIs
a. Adding a Web API Controller
b. Configuring Routes
c. Coding Web API Methods
d. Handling Exceptions
e. Returning Response Codes
f. Injecting Services to Controller

19. Action Return Types in Web API
a. Specific type
b. IActionResult type
c. ActionResult vs IActionResult
d. HttpResults type

20. Handle errors in ASP.NET Core web APIs
a. Developer Exception Page
b. Exception handler
c. Use exceptions to modify the response
d. Validation failure error response
e. Client error response

21. Consuming Web-API Using Swagger
a. Enabling Swagger Support in Web API
b. Swagger UI
c. Testing Web API using Swagger
d. Consuming Web API in .NET Apps
e. Installing Microsoft.AspNet.WebApi.Client
f. Using HttpClient Class
g. Making Asynchronous calls to Web API

h. Calling POST, PUT and DELETE Methods

22. Working with Generative AI

a. What is Generative AI?
b. What are the benefits of Gen AI?
c. Use cases for Gen AI
d. Concerns around Gen AI
e. Using Gen AI in Node JS to build next generation applications
f. Sample .Net Core application with Gen AI